In [9]:
```
# Homework 2 - Problem 1
# This notebook will read in a csv file with data from the 2012 London Summe

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# CSV file is opened and read
df = pd.read_csv("olympics.csv")

# Pull Country column out of the data frame
country_names = tuple(df["Country name"].astype(str))

# Medal Columns Counts.
gold_medals = tuple(df["Gold medals"])
silver_medals = tuple(df["Silver medals"])
bronze_medals = tuple(df["Bronze medals"])

# X-Axis Identity
x_labels = country_names

# Lineup of three bars per country
olympic_means = {
    "Gold" : gold_medals,
    "Silver" : silver_medals,
    "Bronze" : bronze_medals,
}

# Organize countries at the base with medal types a sideways nudge
x = np.arange(len(x_labels))
width = 0.25

# Tracks which medal to place (0 is Gold, 1 is Silver, 2 is Bronze)
multiplier = 0

# Build canvas with a constrained layout
fig, ax = plt.subplots(layout="constrained", figsize=(14, 6))

for medal_type, counts in olympic_means.items():
    # Slides medals sideways so bars don't pile up
    offset = width * multiplier

    # Draw the bars and keep references
    rects = ax.bar(x + offset, counts, width, label=medal_type)

    # Stamp each bar with its count
    ax.bar_label(rects, padding=3, fontsize=8)

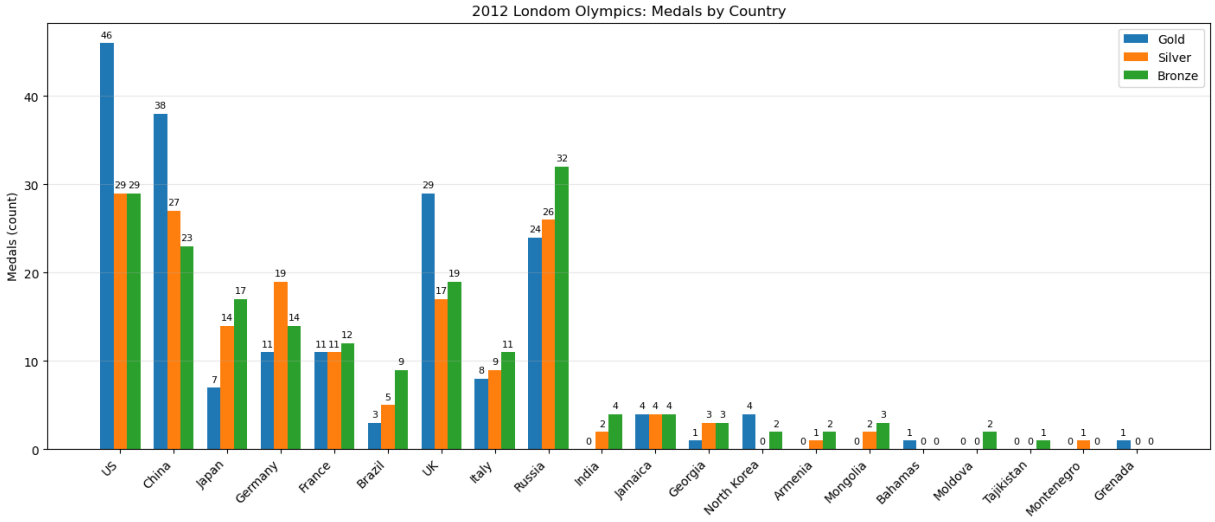
    multiplier += 1

# Official labeling
ax.set_ylabel("Medals (count)")
ax.set_title("2012 London Olympics: Medals by Country")
```

```
# Center tick labels under middle bar of each grouping
middle_label = width * (len(olympic_means) - 1) / 2
ax.set_xticks(x + middle_label, country_names, rotation=45, ha="right")

# Create a legend and a subtle grid
ax.legend(loc="upper right")
ax.grid(axis="y", alpha=0.3)

plt.show()
```




In [10]:
```
# Homework 2 – Problem 3
import numpy as np

np.random.seed(2025)

# Simulate two fair six-sided dice with 1,000 throws
firstDie = np.random.randint(1, 7, size=1000)
secondDie = np.random.randint(1, 7, size=1000)

# Event E: sum equals 8
event_count = np.sum((firstDie + secondDie) == 8)
empirical_frequency = event_count / 1000

print("Empirical Frequency: ", empirical_frequency * 100, "%")
```

```
Empirical Frequency:  15.0 %
```

In [4]:
```
# Homework 2 – Problem 4
import numpy as np

np.random.seed(2025)

# Frequencies list
freq_list = []

# Make 10 runs
for run in range(10):
    firstDie = np.random.randint(1, 7, size=1000)
    secondDie = np.random.randint(1, 7, size=1000)
```

```
    # Event E: sum equals 8
    empirical_frequency = np.mean((firstDie + secondDie) == 8)
    freq_list.append(empirical_frequency)

for run, freq in enumerate(freq_list, 1):
    print(f"Empirical Frequency {run}: {freq*100:.1f}%")
```

```
Empirical Frequency 1: 15.0%
Empirical Frequency 2: 14.7%
Empirical Frequency 3: 14.3%
Empirical Frequency 4: 13.5%
Empirical Frequency 5: 15.2%
Empirical Frequency 6: 12.5%
Empirical Frequency 7: 15.2%
Empirical Frequency 8: 15.0%
Empirical Frequency 9: 14.7%
Empirical Frequency 10: 14.4%
```

In [5]:
```
# Homework 2 – Problem 5
import numpy as np

np.random.seed(2025)

# Frequencies list
freq_list = []

# Make 10 runs
for run in range(10):
    firstDie = np.random.randint(1, 7, size=10000)
    secondDie = np.random.randint(1, 7, size=10000)

    # Event E: sum equals 8
    empirical_frequency = np.mean((firstDie + secondDie) == 8)
    freq_list.append(empirical_frequency)

for run, freq in enumerate(freq_list, 1):
    print(f"Empirical Frequency {run}: {freq*100:.2f}%")
```

```
Empirical Frequency 1: 13.75%
Empirical Frequency 2: 13.92%
Empirical Frequency 3: 13.76%
Empirical Frequency 4: 14.38%
Empirical Frequency 5: 13.70%
Empirical Frequency 6: 14.13%
Empirical Frequency 7: 14.17%
Empirical Frequency 8: 13.76%
Empirical Frequency 9: 14.17%
Empirical Frequency 10: 13.10%
```

In [ ]: